



PAMIBIA
UNIVERSITY
OF SCIENCE AND
TECHNOLOGY

**FACULTY OF COMPUTING AND INFORMATICS
DEPARTMENT OF SOFTWARE ENGINEERING**

QUALIFICATION: BACHELOR OF COMPUTER SCIENCE (SOFTWARE DEVELOPMENT)	
QUALIFICATION CODE: 07BACS	NQF LEVEL: 7
COURSE: ARTIFICIAL INTELLIGENCE	COURSE CODE: ARI711S
ASSESSMENT: Group Project	TOTAL MARKS: 100
RELEASE DATE: 1 st APRIL 2026	DUE DATE: 30 APRIL 2026 @23:59

INSTRUCTION TO STUDENTS:

1. Read all the questions, passages, scenarios, etc., carefully before answering.
2. This project is to be completed in groups of 5–6 students.
3. Avoid copying codes directly from ChatGPT and others; projects with copied codes will be penalised.
4. Host your group code on a well-structured GitHub repository with an appropriate readme and copyright file (read more on [best practices for repositories](#)).
5. Register your **cool group name** and members by clicking the [Google Form link here](#) under the group project sheet.
6. Each group member must make individual commits to the repository.
7. For the final submission, put your code into a single Python notebook, using Python Markdown to explain your steps and findings ([here is a Markdown cheat sheet to guide you](#)). Make sure all your solutions and test cases are displayed in the notebook.
8. As a final submission, include a link to the PDF version of your Python notebook and to your GitHub repository, and remember to add **naftalindeapo** as a contributor.

Part 1: Search Algorithms: Flight Connections Between Cities

[30 Marks]

Write a program that determines how many flight connections apart two cities are.

```
$ python flights.py data
Loading data...
Data loaded.
City: Windhoek
City: Cairo
3 flight connections.
1: Windhoek to Johannesburg
2: Johannesburg to Nairobi
3: Nairobi to Cairo
```

Inspired by shortest-path search problems, this problem explores air-route networks. In this problem, we are interested in finding the shortest path between any two cities by choosing a sequence of flights connecting them. For example, the shortest path between Windhoek and Cairo is 3: Windhoek is connected to Johannesburg by a flight route, Johannesburg is connected to Nairobi by a flight route, and Nairobi is connected to Cairo by a flight route.

Understanding the data

For this problem, you will use the dataset provided [here](#). Each dataset consists of three CSV files: `cities.csv`, `flights.csv`, and `airlines.csv`. Opening the `cities.csv` file, you will see that each city has a `city_id` corresponding to its unique ID, as well as a `city_name` and `country`. In the `flights.csv` file, you will see that each flight has a `flight_id`, together with a `source_city_id`, a `destination_city_id`, and an `airline_id`. Each row, therefore, describes a direct route from one city to another. The `airlines.csv` file provides information about the airline operating each route. Each airline has an `airline_id` and an `airline_name`.

You must write a Python program that determines how many flight connections exist between two cities based on the available direct flight routes. In your program, you may define several data structures to store information from the CSV files. For example, you may use a dictionary to look up a city by its name by mapping city names to a set of corresponding `city_ids`. Another dictionary may map each `city_id` to another dictionary containing the city's name, country, and the set of all outgoing flights from that city. Lastly, you may use a dictionary that maps each `airline_id` to the airline's name.

Specification

In your program, there must be a `shortest_path` function that returns the shortest path from the city with the ID source to the city with the ID target.

- Assuming there is a path from the source to the target, your function should return a list, where each list item is the next (`flight_id`, `city_id`) pair in the path from the source to the target. Each pair should be a tuple of two strings.
 - For example, if the return value of `shortest_path` were `[(101, 4), (205, 7)]`, that would mean that the source city is connected by flight 101 to city 4, and city 4 is connected by flight 205 to city 7, where city 7 is the target.

- If there are multiple paths of minimum length from the source to the target, your function may return any of them.
- If there is no possible path between two cities, your function should return `None`.
- You must also have the `neighbors_for_city` function that you can call, which accepts a `city_id` as input and returns a set of `(flight_id, city_id)` pairs for all cities directly reachable from the given city.

You may write additional functions and/or import other Python standard library modules.

Hint

- While the implementation of search in lecture checks for a goal when a node is removed from the frontier, you may improve the efficiency of your search by checking for a goal as nodes are added to the frontier: if you detect a goal node, there is no need to add it to the frontier; you may simply return the solution immediately.
- You are welcome to borrow and adapt any code from the lecture examples. For example, you may use the lecture implementations for `Node`, `StackFrontier`, and `QueueFrontier`.

Part 2: Optimization: The Hospital Shift Scheduler.

[40 Marks]

Write an AI to solve the hospital shift schedule.

You can download different levels of the staff schedule [here](#)

Background

Nurse scheduling is a complex logistics challenge for healthcare facilities. The objective is to assign qualified staff to specific time slots while adhering to hospital policies, labour laws, and individual availability. Because these rules restrict which assignments are valid, this is framed as a Constraint Satisfaction Problem (CSP). Your task is to develop an AI program that generates a valid weekly schedule using advanced search techniques, including Backtracking Search, Inference, and Constraint Propagation.

Problem Formulation

To solve this as a CSP, the problem is defined by:

- Variables (X): Each specific shift slot for a 7-day week (e.g., `Monday_Morning`, `Monday_Afternoon`, `Monday_Night`, ..., `Sunday_Night`).
- Domain (D): The set of all available nurses (e.g., `{Nurse_A, Nurse_B, Nurse_C, ...}`).
- Constraints (C):
 - Unary: A nurse cannot be assigned to a shift if they have pre-approved leave for that day.
 - Binary: To ensure proper rest, a nurse assigned to a `Night` shift cannot be assigned to the `Morning` shift of the very next day.
 - Higher-Order: No nurse can be assigned to more than 5 total shifts within the 7-day schedule.

```

Generating Weekly Schedule for 2026...
Data loaded: 12 staff members available.
Enforcing node consistency...
Running AC-3 algorithm...
Starting backtracking search...

MONDAY:
  Morning:   Kamati R
  Afternoon: Swartbooi I
  Night:     Mudge D
...

FRIDAY:
  Morning:   Kahuure K (Note: Gaweseb T is Off)
  Afternoon: Naruseb J
  Night:     Eigowab S

SATURDAY:
  Morning:   Hanse-Himarwa K (Note: Eigowab S is Off)
  Afternoon: Mwandingi F
  Night:     Beukes D

SUNDAY:
  Morning:   Kamati R (Note: Beukes D is Off)
  Afternoon: Iivula-Ithana P
  Night:     Naruseb J

Schedule Totals:
- Kamati R: 3 shifts
- Beukes D: 3 shifts
- Mudge D: 2 shifts
- (All others under 5 shifts)

Status: Success. All 21 shifts assigned. All constraints satisfied.

```

Specification

You must implement the `Shift_AI_Solver` class in `shift_solver.py`. Your AI must assign nurses to all 21 weekly shifts so that all constraints are satisfied.

Your implementation should include the following components:

1. `enforce_node_consistency()`: Remove any nurses from a shift's domain if they are on leave during that specific time slot.
2. `revise(x, y)`: Make variable `x` arc consistent with variable `y`. If a nurse assigned to shift `x` makes it impossible to find a valid nurse for shift `y` (due to rest requirements), remove that nurse from `x`'s domain.
3. `ac3()`: Use the AC-3 algorithm to enforce arc consistency across the entire schedule before beginning the search process.
4. `select_unassigned_variable(assignment)`: Use the Minimum Remaining Values (MRV) heuristic to choose the next shift to fill (choose the shift with the fewest eligible nurses remaining).
5. `backtrack(assignment)`: The core recursive function that attempts to assign values to variables one by one, backtracking when it hits a dead-end where constraints cannot be met.

Hint

- Representation: Use a dictionary to store the schedule, where keys are the shift names and values are the assigned nurse names.
- Rest Periods: When implementing `revise`, remember that Night shifts only create binary constraints with the `Morning` shifts of the following day.
- Fail Early: Implementing Forward Checking within your backtracking function will help detect if a nurse's 5-shift limit has been reached, pruning the search tree earlier.

Testing your program

To test your program, download three staff files: `staff_small.txt`, `staff_medium.txt`, and `staff_complex.txt` [here](#). Each file contains a pool of nurses with indicated leave days. Zeros (0) or empty fields in these files indicate unassigned slots. All slots must be filled in the final output.

Part 3: Machine Learning: Traffic Sign Recognition

[30 Marks]

Write an AI that identifies which traffic sign appears in a photograph

```
$ python traffic_signs.py gtsrb model.h5
Loading data...
Data loaded.
Training model...
Model saved to model.h5
Evaluating model
Model accuracy: 0.9535
```

Background

As research continues in the development of self-driving cars, one of the key challenges is computer vision: the ability to understand surroundings from digital images. A crucial subtask of this is the automatic recognition of road signs—such as stop signs, speed limits, yield signs, and more.

In this project, your task is to train an AI model using [TensorFlow](#) that can recognize and classify German traffic signs from images using deep learning. You'll work with a real-world dataset, the [German Traffic Sign Recognition Benchmark](#) (GTSRB), which contains over 50,000 images across 43 sign categories.

Getting Started

1. Download the [GTSRB dataset](#) and unzip it.
2. Inside the unzipped folder, ensure the structure contains **43 subfolders** named 0 through 42, each representing a traffic sign category. Inside each subfolder are **JPEG images** of that category.
3. Place the entire folder (e.g., gtsrb/) in the same directory as your traffic.py script.
4. Run the command: `$ pip install tensorflow opencv-python scikit-learn` to install the dependencies `opencv-python` for image processing, `scikit-learn` for ML-related functions, and `TensorFlow` for neural networks.
5. Explore the dataset: Browse the gtsrb directory. Each category contains colored images of varying sizes and orientations. Before training a model, these will need to be preprocessed (resized, normalized, and labelled).

Understanding the problem

Each image needs to be fed into a neural network that outputs a classification. You will:

- Load and preprocess all images in the dataset.
- Resize them to a fixed size (e.g., 30×30 pixels).
- Assign numerical labels based on folder names.
- Split the dataset into training and testing sets.
- Train a convolutional neural network (CNN) to classify the images.
- Evaluate its accuracy on unseen data.
- (Optional) Save the trained model to disk.

Specification

Your task is to write a Python program in a file called `traffic.py` that trains a model to classify traffic signs using supervised learning. Your program should accept the `asdataset` path to command-line input. Optionally, it may accept a second argument: the filename to which the trained model should be saved.

You are responsible for designing and implementing the full pipeline:

- Read and preprocess the dataset.
- Construct and train the neural network.
- Evaluate the model's performance.
- Save the model for later use.

You are encouraged to experiment with different architectures and training strategies to maximise accuracy. Use OpenCV to load and resize images, and TensorFlow/Keras to define and train your model. Try different combinations of convolutional, pooling, dropout, and dense layers.

Testing

Once your model is trained, run it on the test data and report its accuracy and confusion matrix. You must also do some visual inspection by running the saved model on some randomly selected images from the test set and analyse the model's performance in terms of classifying these images. You can also test the model manually by loading `model.h5` and running it on new traffic sign images (optional challenge).

Hint

- Use `cv2.imread` and `cv2.resize(img, (30, 30))` to process images.
- Use `model.evaluate(X_test, y_test, verbose=2)` to print accuracy.
- Normalise the images by dividing pixel values by 255.0.
- Use `categorical_crossentropy` as your loss function and `SoftMax` as the activation function on the output layer.
- Use Google Collaboratory for high-performance computing (T4 or A100 GPUs).

Deliverables

You should submit:

- A Python program that reads the data, trains a model, and evaluates its accuracy.
- A saved model file (e.g., `model.h5`) if specified.
- Include a `README.md` file describing:
 - Your experimental results, such as accuracy, learning curves, confusion matrices, etc.
 - The architecture and hyperparameters you tried
 - What worked and what didn't
 - Your final model structure